

BÁO CÁO THỰC HÀNH

Môn học: Lập trình thiết bị mạng (IoT)

Chủ đề nghiên cứu: Thí nghiệm giả lập hệ thống quản lý thiết bị nối mạng (IoT)

Người hướng dẫn khoa học và đánh giá: Tiến sỹ LÊ TRỌNG NHÂN

Học viên cao học: NGUYỄN XUÂN HIỀN

Mã số học viên: 2470749

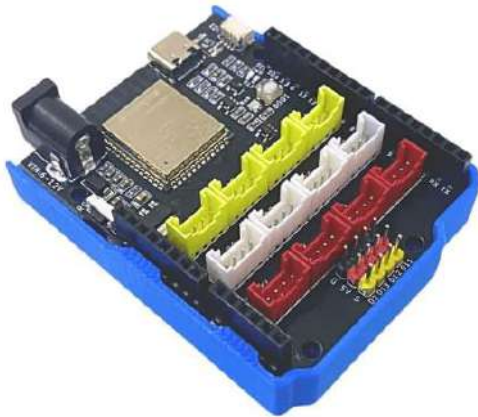
BẢNG ĐÁNH GIÁ TIẾN ĐỘ HOÀN THÀNH NGHIÊN CỨU

STT	Nội dung	Tỷ lệ hoàn thành	Điểm đánh giá	Tham chiếu
1.	Nhiệm vụ 1: Trạng thái tắt / mở đèn LED D13.			
2.	Nhiệm vụ 2: Trạng thái tắt / mở đèn LED RGB.			
3.	Nhiệm vụ 3: DHT20.			
4.	Nhiệm vụ 4: Thiết lập chế độ Web server.			
5.	Nhiệm vụ 5: Thử nghiệm mô hình TinyML.			
6.	Nhiệm vụ 6: Quản trị nền tảng CoreIoT.			
	Điểm đánh giá			

BÁO CÁO THỰC NGHIỆM CHI TIẾT

1. Thiết lập môi trường lập trình cho thiết bị:

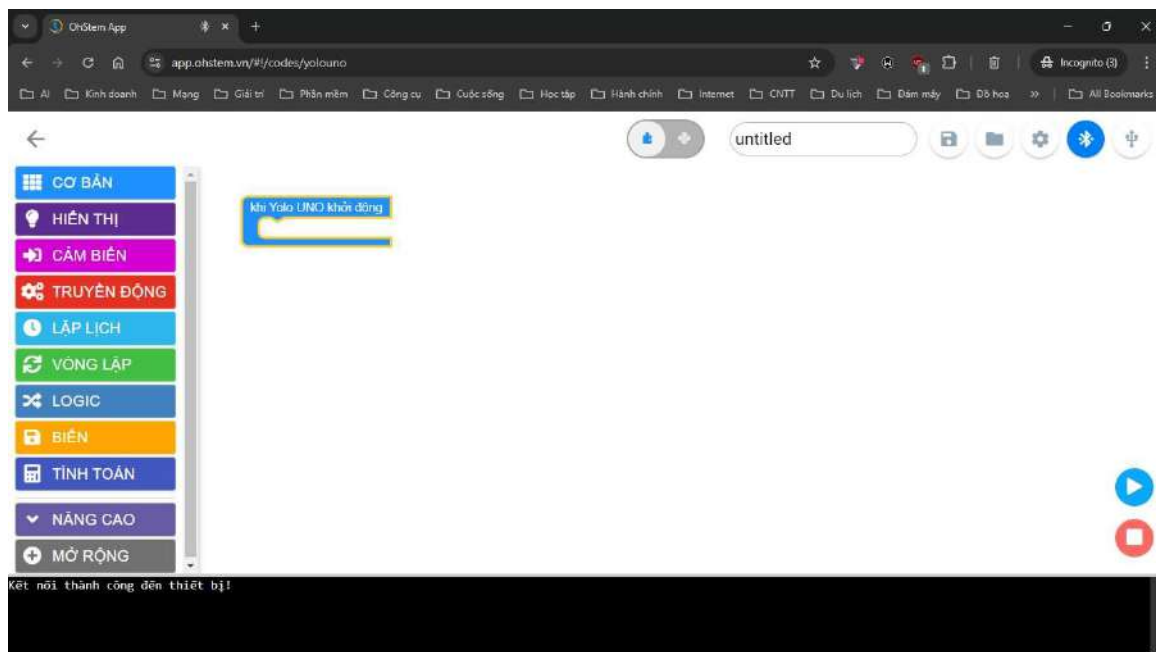
Thiết bị sử dụng: Mạch phát triển Yolo-Uno kết hợp cảm biến nhiệt độ và độ ẩm (DHT20).



Hình 1 - Bo mạch phát triển chính Yolo-Uno Hình 2 - Cảm biến nhiệt độ và độ ẩm DHT20

Trang mạng (website) môi trường lập trình tích hợp: <https://app.ohstem.vn/#!/codes/yolouno>

Kết nối sử dụng: **Bluetooth**. Tên thiết bị: **ohstem-yolo-uno-eab4**



Hình 3 - Môi trường lập trình phát triển - Màn hình thiết kế dạng khối đồ họa

```

1 from yolo_uno import *
2
3 async def setup():
4     print('App started')
5
6
7 async def main():
8     await setup()
9     while True:
10         await asleep_ms(100)
11
12 run_loop(main())
13

```

Kết nối thành công đến thiết bị!

Hình 4 - Môi trường lập trình phát triển - Màn hình mã nguồn Python

2. Triển khai các nhiệm vụ:

Khuyến cáo: Mã nguồn nên được biên tập và mã hoá theo chuẩn ASCII hoặc lưu dưới dạng văn bản thô (plain text), không nên mã hoá theo chuẩn Unicode (tiếng Việt) để tránh gặp lỗi khi biên dịch.

2.1. Nhiệm vụ 1: Trạng thái tắt / mở đèn LED D13.

Yêu cầu nhiệm vụ:

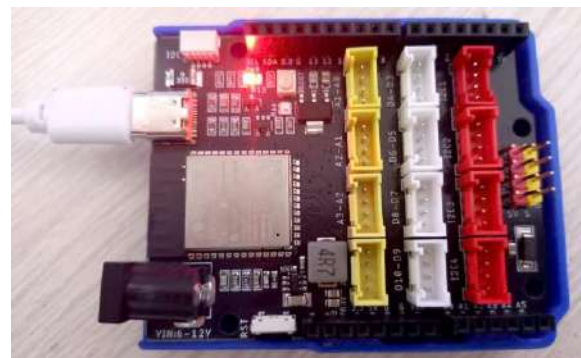
- Tạo một tác vụ chạy độc lập.
- Điều khiển trạng thái mở / tắt (nhấp nháy) cho đèn LED ở chân cắm số 13 (màu mặc định là xanh lá cây). Thời gian chờ chuyển đổi trạng thái là 1.000 ms (hay 1 giây).

Kết quả:

- Tỷ lệ hoàn thành: 100%.
- Hình ảnh minh hoạ.



Hình 5 - Trạng thái mở



Hình 6 - Trạng thái tắt

Mã nguồn thực thi (Xem Phụ lục 3): [Task_100.py](#)

Khuyến nghị: Có thể bổ sung đoạn mã kiểm soát lỗi trong trường hợp khai báo sai chân cắm.

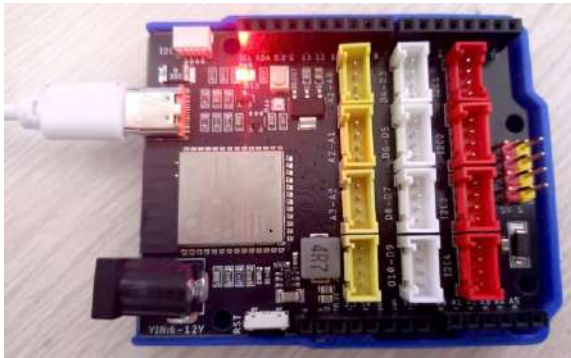
2.2. *Nhiệm vụ 2: Trạng thái tắt / mở đèn LED RGB.*

Yêu cầu nhiệm vụ:

- Tạo một tác vụ chạy độc lập.
- Điều khiển trạng thái mở / tắt (nhấp nháy) và chuyển màu cho đèn LED (Neo). Màu 1 là màu xanh lá (mã màu `#00ff00`), màu 2 là màu vàng (mã màu `#ffff00`) và màu 3 là màu đỏ (mã màu `#ff0000`). Thời gian chờ chuyển đổi trạng thái là 1.000 ms (hay 1 giây).

Kết quả:

- Tỷ lệ hoàn thành: 100%.
- Hình ảnh minh họa.



Hình 7 - Trạng thái tắt



Hình 8 - Trạng thái đèn xanh



Hình 9 - Trạng thái đèn vàng



Hình 10 - Trạng thái đèn đỏ

Mã nguồn thực thi (Xem Phụ lục 3): `Task_200.py`

Khuyến nghị: Có thể bổ sung đoạn mã giới hạn số lần sáng đèn.

2.3. *Nhiệm vụ 3: Đọc dữ liệu từ cảm biến nhiệt độ và độ ẩm.*

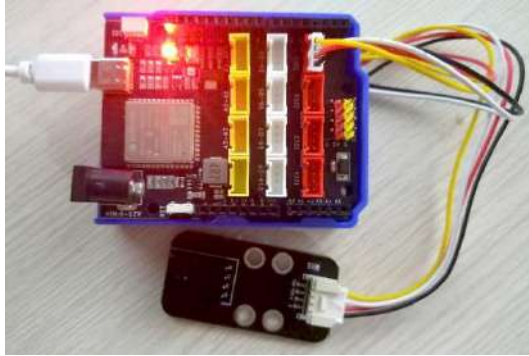
Yêu cầu nhiệm vụ:

- Kết nối thiết bị cảm biến vào bo mạch chính qua cổng I2C.
- Tạo một tác vụ chạy độc lập.
- Đọc giá trị nhiệt độ (temperature) và độ ẩm (humidity) từ cảm biến. Hiển thị giá trị lên màn hình cửa sổ dòng lệnh (terminal). Thời gian chờ cập nhật giá trị là 1.000 ms (hay 1 giây).

Kết quả:

- Tỷ lệ hoàn thành: 100%.

- Hình ảnh minh họa.



Hình 11 - Kết nối cảm biến vào bo mạch chủ.

```

11 while True:
12     await asleep_ms(1000)
13     print("Temperature: ", dht20.temperature(), ". Humidity: ", dht20.humidity())
14 # Define the initial() function to manage task
15 async def initial():
16     # Show application status
17     print('\nRead data from sensor')
18     # Create an independent task or process for DHT_value()
19     # function
20     create_task(DHT_value())
21 # Define the main() function
22 async def main():
23     await initial()
24     while True:
25         await asleep_ms(0)
26 # Run and loop infinitely application by main() function
27 run_loop(main())
    
```

Read data from sensor
 Temperature: 31.6 . Humidity: 78.3
 Temperature: 31.6 . Humidity: 78.3

Hình 12 - Kết quả hiển thị trên cửa sổ dòng lệnh.

Mã nguồn thực thi (Xem Phụ lục 3): [Task_300.py](#)

Khuyến nghị: Tạo đoạn mã kết hợp nhiệm vụ 2 và nhiệm vụ 3 để hiển thị cảnh báo bằng đèn LED (Neo) RGB theo màu tương ứng với các ngưỡng nhiệt độ và độ ẩm của môi trường. Ví dụ: Màu xanh là nhiệt độ thuận lợi cho sức khỏe (mát mẻ), màu vàng là nhiệt độ hơn mức bình thường, màu đỏ nhiệt độ cao có hại cho sức khỏe.

2.4. *Nhiệm vụ 4: Thiết lập máy chủ trang mạng nội bộ.*

Yêu cầu nhiệm vụ:

- Thiết lập và cấu hình máy chủ trang mạng nội bộ (Website) cho mạch chủ với giao thức HTTP (cổng 80).
- Tạo một tác vụ chạy độc lập 1: Đọc giá trị nhiệt độ (temperature) và độ ẩm (humidity) từ cảm biến. Hiển thị giá trị lên màn hình cửa sổ dòng lệnh (terminal). Thời gian chờ cập nhật giá trị là 1.000 ms (hay 1 giây).

- Tạo một tác vụ chạy độc lập 2: Tạo trang chủ (homepage) chuẩn HTML, CSS hiển thị giá trị thu được từ cảm biến kèm theo dấu thời gian (timestamp). Đồng thời hiển thị nút lệnh mở / tắt thiết bị.

Kết quả:

- Tỷ lệ hoàn thành: 0%.

Mã nguồn thực thi: Chưa cập nhật.

Khuyến nghị: Triển khai trên một máy chủ trang mạng nội bộ có cấu hình mạnh hơn (Raspberry Pi 4) và có thể lưu trữ dữ liệu nhiều hơn; đồng thời, thiết lập giao thức bảo mật HTTPS với OpenSSL.

2.5. *Nhiệm vụ 5: Thử nghiệm mô hình TinyML.*

Yêu cầu nhiệm vụ:

- Tạo tập dữ liệu mẫu.
- Triển khai mô hình TinyML cho dữ liệu chuỗi thời gian.

Kết quả:

- Tỷ lệ hoàn thành: 0%.

Mã nguồn thực thi: Chưa cập nhật.

Khuyến nghị: Triển khai mô hình trên một máy tính cận biên (edge computing) nội bộ độc lập có cấu hình mạnh hơn (Raspberry Pi 4).

2.6. *Nhiệm vụ 6: Triển khai nền tảng quản trị thiết bị.*

Yêu cầu nhiệm vụ:

- Thiết lập và cấu hình nền tảng CoreIoT để quản lý thiết bị và lưu trữ dữ liệu.
- Thiết kế bảng điều khiển (dashboard) với các tính năng chính: hiển thị nhiệt độ và độ ẩm hiện tại, biểu đồ biểu diễn biến động nhiệt độ và độ ẩm, nút lệnh mở / tắt thiết bị.

Kết quả:

- Tỷ lệ hoàn thành: 0%.

Mã nguồn thực thi: Chưa cập nhật.

Khuyến nghị: Triển khai nền tảng quản lý thiết bị mã nguồn mở (thay thế cho CoreIoT) trên máy chủ nội bộ.

PHỤ LỤC 1

LỊCH SỬ CẬP NHẬT BÁO CÁO

* Phát hành lần đầu. Ngày 20/05/2026. Số trang tài liệu 11.

PHỤ LỤC 2

THÔNG TIN THIẾT BỊ

Tên thiết bị: **Yolo UNO**

Thông số kỹ thuật:

Vi xử lý: **ESP32-S3** Dual Core 240MHz Tensilica, 4MB Flash, 8MB PSRAM.

Kết nối không dây: 2.4GHz Wifi - 802.11b/g/n, Bluetooth 5, BLE + Mesh.

Kết nối có dây: USB type C. Kết nối dữ liệu và cấp nguồn.

Kết nối nguồn: 12V DC 5521.

Điện năng tiêu thụ thực tế (tối đa) qua cổng USB Type C là: **4.8 V ~ 0.07A**.

Nút điều khiển trên mạch: 2 nút nhấn onboard: Reset. Bootloader mode (BOOT0).

Cổng kết nối ngoại vi: 4 x Analog. 4 x Digital. 4 x I2C.

Chân cắm:

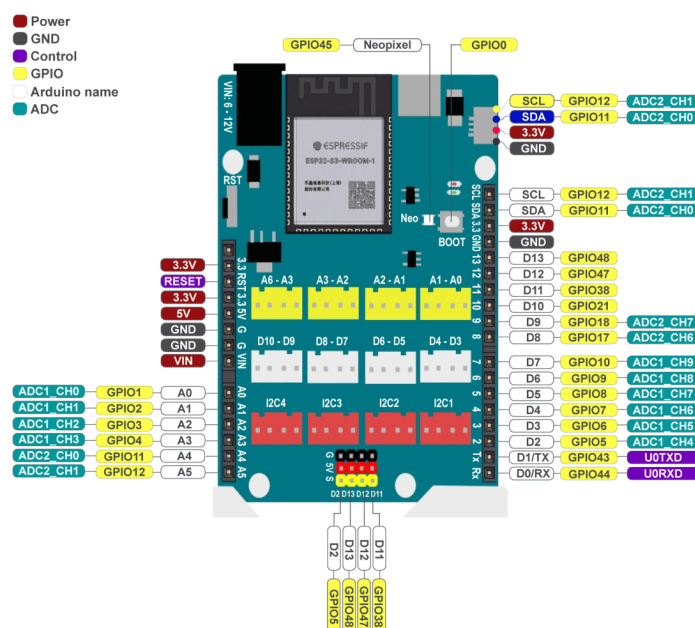
* 4 chân cắm GVS giúp dễ dàng giao tiếp các loại động cơ Servo

* Cổng kết nối chuẩn STEMMA QT/QWIIC connector cho các module I2C

Tích hợp đèn LED báo nguồn, LED chân D13 và LED RGB NeoPixel

Hỗ trợ lập trình bằng khối lệnh kéo thả, Arduino, Micropython hoặc ESP-IDF.

Sơ đồ mạch.



PHỤ LỤC 3

MÃ NGUỒN NGHIÊN CỨU

Liên kết mã nguồn GitHub: https://github.com/hiennx2k4/HCMUT_IoTProgramming.git

Nhiệm vụ 1: Trạng thái tắt / mở đèn LED D13.

Tên tập tin mã nguồn: **Task_100.py**

```
# Declare and import library
# Using library of Yolo:Uno device
from yolo_uno import *
# Using library of pins control
from pins import *
# Declare LED as 13th pin
led_D13 = Pins(D13_PIN)
# Define LED_blink() function to control LED status (ON/OFF)
# Set delay time to 1.000 ms (1 second)
async def LED_blink():
    while True:
        led_D13.write_digital(1)
        await asleep_ms(1000)
        led_D13.write_digital(0)
        await asleep_ms(1000)
# Define the initial() function to manage task
async def initial():
    # Show application status
    print('App started')
    # Create an independent task or process for LED_blink()
    # function
    create_task(LED_blink())
# Define the main() function
async def main():
    await initial()
    while True:
        await asleep_ms(0)
# Run and loop infinitely application by main() function
run_loop(main())
```

Nhiệm vụ 2: Trạng thái tắt / mở đèn LED RGB.Tên tập tin mã nguồn: **Task_200.py**

```
# Declare and import library
# Using library of Yolo:Uno device
from yolo_uno import *
# Define LED_change() function to change LED RGB color
# Set delay time to 1.000 ms (1 second) per action
async def LED_change():
    while True:
        # Set LED color to green
        neopix.show(0, hex_to_rgb('#00ff00'))
        await asleep_ms(1000)
        # Set LED color to yellow
        neopix.show(0, hex_to_rgb('#ffff00'))
        await asleep_ms(1000)
        # Set LED color to red
        neopix.show(0, hex_to_rgb('#ff0000'))
        await asleep_ms(1000)
# Define the initial() function to manage task
async def initial():
    # Show application status
    print('App started')
    # Create an independent task or process for LED_change()
    # function
    create_task(LED_change())
# Define the main() function
async def main():
    await initial()
    while True:
        await asleep_ms(0)
# Run and loop infinitely application by main() function
run_loop(main())
```

Nhiệm vụ 3: Đọc dữ liệu từ cảm biến nhiệt độ và độ ẩm.Tên tập tin mã nguồn: **Task_300.py**

```
# Declare and import library
# Using library of Yolo:Uno device
from yolo_uno import *
# Using library of temperature and humidity sensor
from dht20 import *
# Declare variable for command
dht20 = DHT20()
# Define DHT_value() function to get value from sensor
# Set delay time to 1.000 ms (1 second) per action
async def DHT_value():
    while True:
        await asleep_ms(1000)
        print("Temperature: ",dht20.temperature(),". Humidity: ",
dht20.humidity())
# Define the initial() function to manage task
async def initial():
    # Show application status
    print('\nRead data from sensor')
    # Create an independent task or process for DHT_value()
    # function
    create_task(DHT_value())
# Define the main() function
async def main():
    await initial()
    while True:
        await asleep_ms(0)
# Run and loop infinitely application by main() function
run_loop(main())
```

PHỤ LỤC 4

CẤU HÌNH NỀN TẢNG QUẢN LÝ THIẾT BỊ COREIOT

Nền tảng quản trị thiết bị mạng Core IoT:

Thông tin thiết lập tài khoản quản trị:

Tên hồ sơ: ***nxhien.sdh242@hcmut.edu.vn***

Hệ thống đơn vị đo lường: ***Metric***.

Thông tin thiết lập thiết bị giả lập:

Tên thiết bị: 00100100001. Quy ước đặt tên mã hoá (từ trái sang phải): **bra** **dep** **devic**

bra: số thứ tự chi nhánh / nhà máy trực thuộc doanh nghiệp (gồm 3 ký tự số từ 0-9).

dep: số thứ tự phòng ban / đơn vị nội bộ thuộc chi nhánh / nhà máy (gồm 3 ký tự số từ 0-9).

devic: số thiết bị được lắp đặt trong phòng ban / đơn vị nội bộ (gồm 5 ký tự số từ 0-9).

Mã truy cập (Token): **hinzvhaa8i47dvne24pp**.

Thông tin mô tả: Thiết bị giả lập cảm biến nhiệt độ, độ ẩm.

Mục đích: Kiểm tra kết nối dữ liệu giữa thiết bị và nền tảng quản lý.

PHỤ LỤC 5

TÀI LIỆU VÀ NGUỒN THAM KHẢO

Liên kết tài liệu về nền tảng CoreIoT: <https://coreiot.io/docs/>

Liên kết tài liệu về mạch YoloUno:

https://docs.ohstem.vn/en/latest/yolo_uno/yolo_uno_khoi_lenh.html